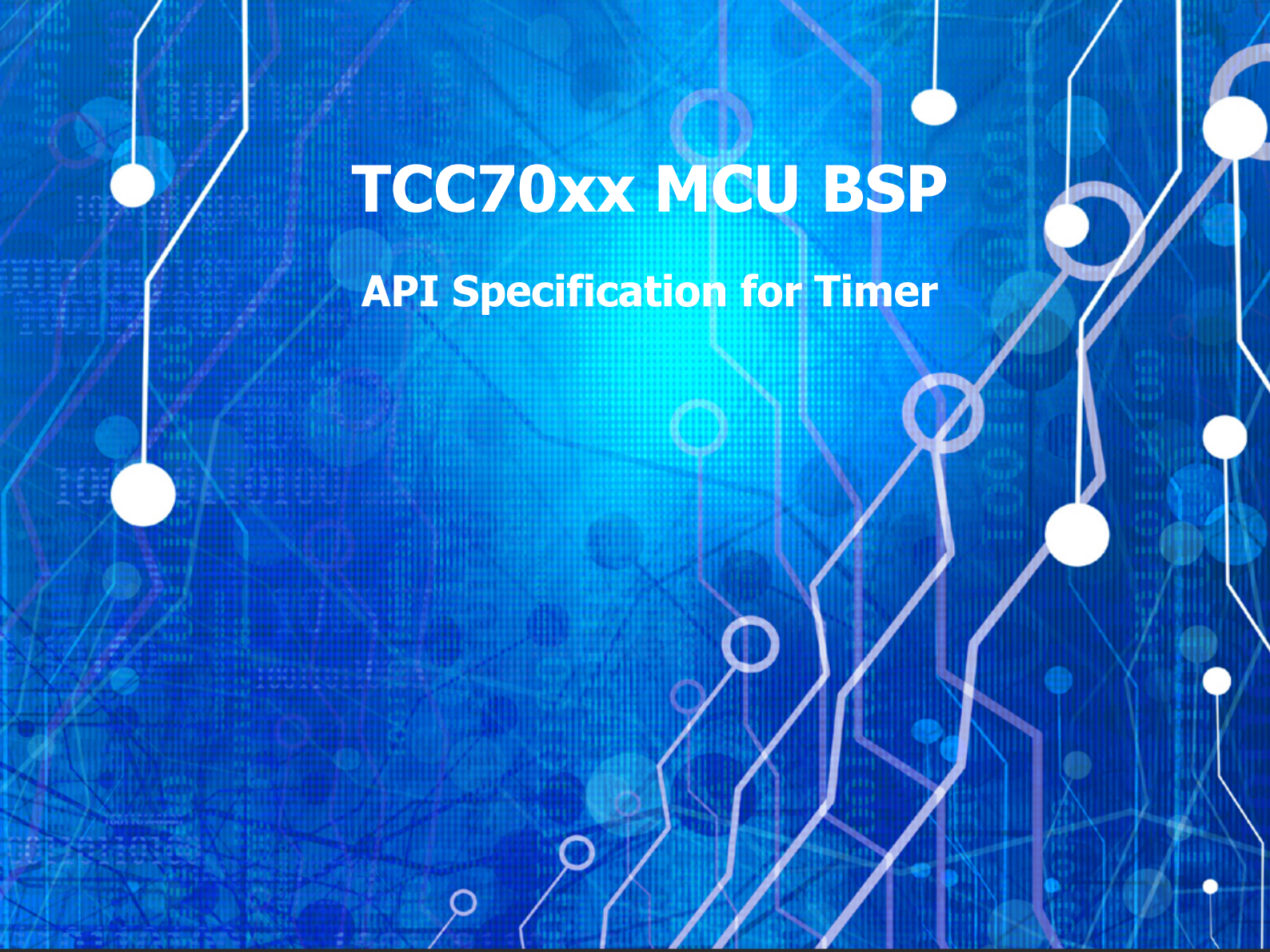




TCC70xx MCU BSP

API Specification for Timer

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Enumerations	5
4.1	TIMERStartMode_t.....	5
4.2	TIMERChannel_t.....	6
4.3	TIMERCounterMode_t.....	7
4.4	TIMEROpMode_t.....	8
4.5	TIMERErrorCode_t.....	9
5	Constants.....	10
5.1.1	Function Indexes.....	10
6	Structures	11
6.1	TIMERConfig_t.....	11
7	Callback Functions	12
7.1	TIMERHandler.....	12
8	APIs	13
8.1	TIMER_InterruptClear	13
8.2	TIMER_Enable.....	14
8.3	TIMER_EnableWithMode	15
8.4	TIMER_EnableWithCfg	16
8.5	TIMER_Disable.....	17
8.6	TIMER_Init	18
9	How to use Timer Driver.....	19
9.1	Starting Timer	19
9.2	Stopping Timer.....	19
10	References	20
11	Revision History	21
	Rev. 0.10: 2021-09-03	21

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the timer device driver (hereinafter referred to as timer driver) of TCC70xx MCU BSP. For more detailed information on timer in the TCC70xx MCU Sub-system, refer to Chapter 19 Timer in "*TCC70xx Full Specification*"[1].

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

TCO	Timer Counter Output
-----	----------------------

3 SOURCE FILES

3.1 Location of Source Files

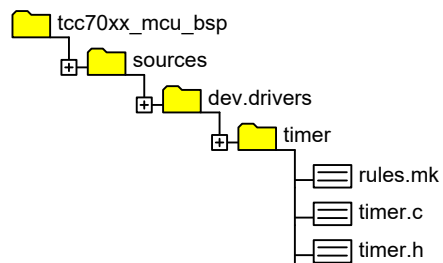


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `rules.mk`
 - Rules used for build
- `timer.c`
 - Timer driver source
- `timer.h`
 - Timer driver API header

4 ENUMERATIONS

4.1 TIMERStartMode_t

Enumeration variables to select whether the start value of counter is 0 or not

Declaration

```
typedef enum TIMERStartMode
{
    TIMER_START_MAINCNT    = 0x0UL,
    TIMER_START_ZERO       = 0x1UL,
} TIMERStartMode_t;
```

Description

Value	Description
TIMER_START_MAINCNT	Counter is forced to count from “LOAD_VALUE” to 0xFFFFFFFF.
TIMER_START_ZERO	Counter is forced to count from 0 to “LOAD_VALUE”.

Remark

None

4.2 TIMERChannel_t

Enumeration variables for each timer module

Declaration

```
typedef enum TIMERChannel
{
    TIMER_CH_0    = 0UL,
    TIMER_CH_1    = 1UL,
    TIMER_CH_2    = 2UL,
    TIMER_CH_3    = 3UL,
    TIMER_CH_4    = 4UL,
    TIMER_CH_5    = 5UL,
    TIMER_CH_6    = 6UL,
    TIMER_CH_7    = 7UL,
    TIMER_CH_8    = 8UL,
    TIMER_CH_9    = 9UL,
    TIMER_CH_MAX  = 10UL,
} TIMERChannel_t;
```

Description

Value	Description
<i>TIMER_CH_XXX</i>	<i>Identifier of each timer module</i>

Remark

None

4.3 TIMERCounterMode_t

Enumeration variables for timer counter mode

Declaration

```
typedef enum TIMERCounterMode
{
    TIMER_COUNTER_MAIN          = 0UL,
    TIMER_COUNTER_COMP0         = 1UL,
    TIMER_COUNTER_COMP1         = 2UL,
    TIMER_COUNTER_SMALL_COMP    = 3UL,
} TIMERCounterMode_t;
```

Description

Value	Description
<i>TIMER_COUNTER_MAIN</i>	Re-load counter when the counter value is matched to <i>MAIN_CNT_LDV</i> or <i>0xFFFFFFFF</i> .
<i>TIMER_COUNTER_COMP0</i>	Re-load counter when the counter value is matched to <i>CMP_VAL0</i> . The value loaded to the counter is selected according to the <i>LD_ZERO</i> setting value.
<i>TIMER_COUNTER_COMP1</i>	Re-load counter when the counter value is matched to <i>CMP_VAL1</i> . The value loaded to the counter is selected according to the <i>LD_ZERO</i> setting value.
<i>TIMER_COUNTER_SMALL_COMP</i>	<i>TIMER_COUNTER_COMP0</i> or <i>TIMER_COUNTER_COMP1</i> (less value)

Remark

None

4.4 TIMEROpMode_t

Enumeration variables for timer operation mode. If the interrupt signal is used several times, it is recommended to use timer in free running mode. In not free running mode, setting is required after interrupt signal.

Declaration

```
typedef enum TIMEROperationMode
{
    TIMER_OP_FREERUN      = 0UL,
    TIMER_OP_ONESHOT
} TIMEROpMode_t;
```

Description

Value	Description
<i>TIMER_OP_FREERUN</i>	<i>Free running mode. Timer continuously counts from start value to interrupt timing value, even though an interrupt is not cleared.</i>
<i>TIMER_OP_ONESHOT</i>	<i>Not free running mode</i>

Remark

None

4.5 TIMERErrorCode_t

Enumeration variables for execution error codes of timer user interface API

Declaration

```
typedef enum TIMERErrorCode
{
    TIMER_ERR_ALREADY_DONE           = (uint32)SAL_DRVID_TMR,
    TIMER_ERR_SET_CLOCK              = (uint32)SAL_DRVID_TMR + 1UL,
    TIMER_ERR_NOT_INITIALIZED        = (uint32)SAL_DRVID_TMR + 2UL,
    TIMER_ERR_INVALID_PARAM          = (uint32)SAL_DRVID_TMR + 3UL,
    TIMER_ERR_OUTOF_RANGE_CHANNEL    = (uint32)SAL_DRVID_TMR + 4UL,
    TIMER_ERR_OUTOF_COMPARATOR_0     = (uint32)SAL_DRVID_TMR + 5UL,
    TIMER_ERR_OUTOF_COMPARATOR_1     = (uint32)SAL_DRVID_TMR + 6UL,
    TIMER_ERR_NOT_SUPPORT_MODE       = (uint32)SAL_DRVID_TMR + 7UL,
    TIMER_ERR_INVALID_TIME           = (uint32)SAL_DRVID_TMR + 8UL,
    TIMER_ERR_OUTOF_MAINCNT          = (uint32)SAL_DRVID_TMR + 9UL
} TIMERErrorCode_t;
```

Description

Value	Description
<i>TIMER_ERR_ALREADY_DONE</i>	<i>A job is re-executed.</i>
<i>TIMER_ERR_SET_CLOCK</i>	<i>A failure occurred when setting the specific clock.</i>
<i>TIMER_ERR_NOT_INITIALIZED</i>	<i>Timer module is not initialized.</i>
<i>TIMER_ERR_INVALID_PARAM</i>	<i>Invalid parameters are passed.</i>
<i>TIMER_ERR_OUTOF_RANGE_CHANNEL</i>	<i>Index of module does not exist.</i>
<i>TIMER_ERR_OUTOF_COMPARATOR_0</i>	<i>Configuration of comparator0 is not valid, that is out of range.</i>
<i>TIMER_ERR_OUTOF_COMPARATOR_1</i>	<i>Configuration of comaprator1 is not valid, that is out of range.</i>
<i>TIMER_ERR_NOT_SUPPORT_MODE</i>	<i>Mode is not supported.</i>
<i>TIMER_ERR_INVALID_TIME</i>	<i>Time to be configured is not valid.</i>
<i>TIMER_ERR_OUTOF_MAINCNT</i>	<i>Counter value of main counter is out of range.</i>

Remark

None

5 CONSTANTS

5.1.1 Function Indexes

Indexing constants correspond to each API function. These indexes are used for debugging or informing information of Timer API Function.

Declaration

```
#define TIMER_API_COMMON_INDEX    (0x0)
#define TIMER_API_ENABLE          (TIMER_API_COMMON_INDEX + 0)
#define TIMER_API_ENABLE_MODE     (TIMER_API_COMMON_INDEX + 1)
#define TIMER_API_ENABLE_CFG      (TIMER_API_COMMON_INDEX + 2)
#define TIMER_API_DISABLE         (TIMER_API_COMMON_INDEX + 3)
#define TIMER_API_INIT            (TIMER_API_COMMON_INDEX + 4)
#define TIMER_API_CLEAR_IRQ       (TIMER_API_COMMON_INDEX + 5)
```

Description

<i>Value</i>	<i>Description</i>
TIMER_API_COMMON_INDEX	<i>Starting index of timer module</i>
TIMER_API_ENABLE	<i>Index of TIMER_Enable() function</i>
TIMER_API_ENABLE_MODE	<i>Index of TIMER_EnableWithMode() function</i>
TIMER_API_ENABLE_CFG	<i>Index of TIMER_EnableWithCfg() function</i>
TIMER_API_DISABLE	<i>Index of TIMER_Disable() function</i>
TIMER_API_INIT	<i>Index of TIMER_Init() function</i>
TIMER_API_CLEAR_IRQ	<i>Index of TIMER_InterruptClear() function</i>

Remark

None

6 STRUCTURES

6.1 TIMERConfig_t

Structure variables for configuring Timer module

Declaration

```
typedef struct TIMERCfgTable
{
    TIMERChannel_t      ctChannel;
    TIMERStartMode_t    ctStartMode;
    TIMEROpMode_t       ctOpMode;
    TIMERCounterMode_t  ctCounterMode;
    uint32              ctMainValueUsec;
    uint32              ctCmp0ValueUsec;
    uint32              ctCmp1ValueUsec;
    TIMERHandler         fnHandler;
    void *              pArgs;
} TIMERConfig_t;
```

Description

Value	Description
<i>ctChannel</i>	Each channel for Timer module-N (0 to 9). Refer to TIMERChannel t .
<i>ctStartMode</i>	Start mode. Refer to TIMERStartMode t .
<i>ctOpMode</i>	Operation mode. Refer to TIMEROpMode t .
<i>ctCounterMode</i>	Counter mode. Refer to TIMERCounterMode t .
<i>ctMainValueUsec</i>	Value of main counter
<i>ctCmp0ValueUsec</i>	Counter comparing value, Counter Compare Value-0 register
<i>ctCmp1ValueUsec</i>	Counter comparing value, Counter Compare Value-1 register
<i>fnHandler</i>	External handler that you need to register. Refer to TIMERHandler .
<i>pArgs</i>	Pointer of configuration structure table

Remark

None

7 CALLBACK FUNCTIONS

7.1 TIMERHandler

When the counter value is matched to the configured value and the interrupt signal is generated, the registered callback function is invoked.

Declaration

```
typedef sint32 (* TIMERHandler)
(
    TIMERChannel_t    uiChannel,
    void *            pArgs
);
```

Parameters

Value	Description
<i>uiChannel</i>	Index for Timer module. Refer to TIMERChannel_t .
<i>pArgs</i>	Additional arguments for parameters

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

8 APIs

8.1 TIMER_InterruptClear

Clears the interrupt signal that occurs when certain conditions are met. This function is automatically called to deal with timer handler, so you do not need to call this function.

Declaration

```
SALRetCode_t TIMER_InterruptClear
(
    TIMERChannel_t    uiChannel
);
```

Parameters

Value	Description
<i>uiChannel</i>	<i>[in]</i> Specific channel for timer module. Refer to TIMERChannel_t .

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

8.2 TIMER_Enable

Enables the specific timer module with *TIMER_OP_FREERUN*, *TIMER_START_ZERO*, and *TIMER_COUNTER_COMP0*.

Declaration

```
SALRetCode_t TIMER_Enable
(
    TIMERChannel_t    uiChannel,
    uint32            uiUSec,
    TIMERHandler      fnHandler,
    void *             pArgs
);
```

Parameters

Value	Description
<i>uiChannel</i>	[in] Specific channel for timer module. Refer to TIMERChannel_t .
<i>uiUSec</i>	[in] Micro-second that you want to set for interrupt signal
<i>fnHandler</i>	[in] External handler that you need to register. Refer to TIMERHandler .
<i>pArgs</i>	[in] Pointer of configuration structure table

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

8.3 TIMER_EnableWithMode

Enables the specific timer module with **TIMER_START_ZERO** and **TIMER_COUNTER_COMP0**.

Declaration

```
SALRetCode_t TIMER_EnableWithMode
(
    TIMERChannel_t    uiChannel,
    uint32            uiUsec,
    TIMEROpMode_t     uiOpMode,
    TIMERHandler      fnHandler,
    void *            pArgs
);
```

Parameters

Value	Description
<i>uiChannel</i>	<i>[in] Specific channel for timer module. Refer to TIMERChannel_t.</i>
<i>uiUsec</i>	<i>[in] Micro-second that you want to set for interrupt signal.</i>
<i>uiOpMode</i>	<i>[in] Operation mode, Free running or One-time running (Not free running mode). Refer to TIMEROpMode_t.</i>
<i>fnHandler</i>	<i>[in] External handler that you need to register. Refer to TIMERHandler.</i>
<i>pArgs</i>	<i>[in] Pointer of configuration structure table</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

8.4 TIMER_EnableWithCfg

Enables the specific timer module with configuration structure.

Declaration

```
SALRetCode_t TIMER_EnableWithCfg  
(  
    TIMERConfig_t *    pCfg  
);
```

Parameters

Value	Description
<i>pCfg</i>	<i>[in]</i> Configuration structure with timer module information. Refer to TIMERConfig t .

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

8.5 TIMER_Disable

Disables the specific timer module.

Declaration

```
SALRetCode_t TIMER_Disable
(
    TIMERChannel_t    uiChannel
);
```

Parameters

Value	Description
uiChannel	[in] Specific channel for timer module. Refer to TIMERChannel_t .

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.6 TIMER_Init

Initializes clock and registers for all channels.

Declaration

```
SALRetCode_t TIMER_Init  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

9 HOW TO USE TIMER DRIVER

The timer driver calls the GIC driver API to enable an interrupt service routine. Refer to GIC IntSrcEn function of the GIC driver in “TCC70xx MCU BSP-API Specification for Interrupt Controller” [2].

9.1 Starting Timer

```
#define TIMER_TESTING_RIGHT_TIME    (3 * 1000 * 1000)    // 3 sec

static TIMERConfig_t                mTimerCfg;

sint32 TIMER_MainHandler(TIMERChannel_t iChannel, void * args)
{
    return SAL_RET_SUCCESS;
}

{
    mTimerCfg.ctChannel      = TIMER_CH_1;
    mTimerCfg.ctStartMode    = TIMER_START_MAINCNT;
    mTimerCfg.ctOpMode       = TIMER_OP_FREERUN;
    mTimerCfg.ctCounterMode  = TIMER_COUNTER_MAIN;
    mTimerCfg.ctCmp0ValueUsec = 0;
    mTimerCfg.ctCmp1ValueUsec = 0;
    mTimerCfg.ctMainValueUsec = TIMER_TESTING_RIGHT_TIME;
    mTimerCfg.fnHandler      = TIMER_MainHandler;
    mTimerCfg.pArgs          = NULL;

    (void)TIMER_EnableWithCfg(&mTimerCfg);
    GIC_IntSrcEn(GIC_TIMER_0 + mTimerCfg.ctChannel);
}
```

9.2 Stopping Timer

```
{
    TIMER_Disable(mTimerCfg.ctChannel);
}
```

10 REFERENCES

- [1] TC70xx Full Specification
- [2] TCC70xx MCU BSP-API Specification for Interrupt Controller
- [3] Contact Telechips for more details: sales@telechips.com

11 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."